Détection du stress en utilisant des signaux EDa, HRV et des données contextuelles

Explication des codes créés en stage ST01 Master

Spécialité : M1 biomécanique et bioingénierie

Fait Par : Abdelwahab OUAZOUL

Encadré par :

- **GHIATT Kawtar**
- **ZALC Vincent**
- **ISTRATE Dan**
- **HO BA THO Marie Christine**

Table des matières

Table des matières 1
Contexte général 2
Organisation du document 3
I. Exigences pour le code 3
I.1. Bibliothèques 4
I.2. Codes prérequis 4
II. Variables utilisées 4
II.1. Variables du signal EDA 4
II.1.1. Explication du code 4
Fonction (feature engineering.py, ligne 13 à 16) 4
II.1.2. Pourquoi cvxEDA 5
II.2. Explication des caractéristiques 5
II.2.1. Caractéristiques statistiques 5
II.2.2. Caractéristiques supplémentaires du dataset (non physiologiques) 7
Tableau 3 : Supplémentaires 7
II.2.3. Catégories physiologiques des caractéristiques (Features) 7
II.3. Pourquoi utiliser 37 caractéristiques ? 8
III Paramètres du modèle Random Forest (Classification et validation) 9
III.1. Pipeline 9
III.2. n_estimators = 300 10
III.3 class_weight = "balanced" 10
III.4. random_state = 42 12
IIII.5. n_jobs = -1 12
IV. Validation 12
Exemple de division 15
V. Bilan 15
Bibliographie 16

Contexte général

“Random forest” est un modèle d’apprentissage automatique utilisé pour la **classification** et la **prévision**. Pour entraîner des algorithmes d’apprentissage automatique et des modèles d’intelligence artificielle, il est essentiel de disposer d’une grande quantité de **données de haute qualité** afin d’assurer une collecte de données efficace.

Les données relatives aux performances d’un système sont essentielles pour **améliorer les algorithmes**, optimiser l’efficacité des logiciels et du matériel, analyser le comportement des utilisateurs, permettre l’identification de modèles, la prise de décision, la modélisation prédictive et la résolution de problèmes. Tout cela conduit finalement à une **meilleure efficacité et une plus grande précision**.

L’intégration de différentes méthodes de collecte et de traitement des données améliore la **précision** et favorise **l’innovation dans la résolution de problèmes**. L’utilisation de méthodologies variées dans la recherche interdisciplinaire simplifie le processus de recherche, encourage l’innovation et permet d’appliquer les résultats de l’analyse de données à la **reconnaissance de motifs, à la prise de décision, à la modélisation prédictive et à la résolution de problèmes**. Cette approche favorise également l’innovation dans la recherche interdisciplinaire.

Cette technique repose sur le concept des **arbres de décision** : elle construit un ensemble d’arbres de décision et combine leurs résultats pour produire la **prédiction finale**. Chaque arbre de décision dans le “Random Forest” est construit à partir de **sous-ensembles aléatoires de données**, et chaque arbre est entraîné sur une partie de l’ensemble complet de données. Ensuite, les résultats de tous les arbres sont **agrégés** afin d’obtenir la prévision finale.

L’un des avantages des forêts aléatoires est leur capacité à **gérer des données déséquilibrées** ainsi que des variables comportant des **valeurs manquantes**. Elles permettent également de réduire le problème de **sélection arbitraire des variables** observé dans certains modèles alternatifs. De plus, les forêts aléatoires limitent le **surapprentissage (overfitting)** en entraînant plusieurs arbres de décision sur des

sous-ensembles aléatoires de données, ce qui améliore leur capacité à **généraliser à de nouvelles données**.

Les forêts aléatoires sont largement considérées comme l'une des techniques **les plus efficaces et les plus puissantes** dans le domaine de l'apprentissage automatique. Elles sont largement utilisées dans diverses applications telles que la **classification automatique, la prévision de données et l'apprentissage supervisé [3]**.

Organisation du document

Ce document explique deux codes (Classification et Validation.py, Feature engineering.py) à chaque fonction expliquée une référence de quel code se situe est mise à côté. Sur la première étape j'ai cité les exigences pour établir ce code.

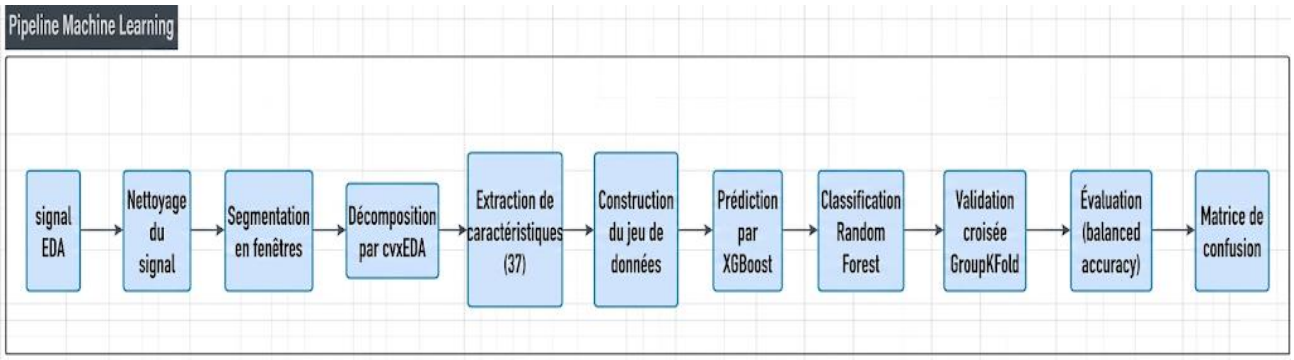


FIGURE 1 : PIPELINE

I. Exigences pour le code

I.1. Bibliothèques

```
import pandas as pd
import numpy as np
from cvxEDA import cvxEDA
import xgboost as xgb
import os
import glob
```

FIGURE 2 : FEATURE ENGENEERING

```
import numpy as np
import pandas as pd
from pathlib import Path
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GroupKFold
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.metrics import balanced_accuracy_score, confusion_matrix
```

FIGURE 3 : CLASSIFIACTION ET VALIDATION

I.2. Codes prérequis

- Fichier de détection des artefacts [1]
- Bibliothèque cvxEDA pour décomposition du signal EDA [2]

II. Variables utilisées

II.1. Variables du signal EDA

II.1.1. Explication du code

Fonction (feature engineering.py, ligne 13 à 16)

```
def extract_features(y, sr=4):
    y = np.asarray(y, dtype=float)
    [r, p, t, l, d, e, obj] = cvxEDA(y, 1. / sr)
    feats = {}
```

FIGURE 4 : EXTRACT FEATURES

A travers la fonction extract_features le signal EDA est décomposé à l'aide de l'algorithme cvxEDA en 7 sorties correspondant aux différentes composantes estimées du signal EDA.

Ils sont représentés comme suit :

Variable	Signification	Type
r	composante phasique	physiologique
p	activité nerveuse sudomotrice	physiologique (théorique)

t	composante tonique	physiologique
l	dérive lente	correction du modèle
d	coefficients spline tonic	paramètres mathématiques
e	bruit / résidu	erreur
obj	fonction objectif	optimisation

TABLEAU 1 : CVXEDA

II.1.2. Pourquoi cvxEDA

Le modèle cvxEDA repose sur un modèle physiologique du système sudomoteur, une décomposition phasique, tonique et une optimisation convexe régularisée. Ce qui en fait l'une des, méthodes les plus utilisés pour analyser les signaux EDA dans les études de **stress, émotion et charge cognitive**.

II.2. Explication des caractéristiques

Les caractéristiques sont calculées dans la fonction `extract_features()` du script.

II.2.1. Caractéristiques statistiques

Feature	Signal	Type	Signification	Raison du choix
phasic_mean	phasic	statistique	moyenne du signal phasique	niveau moyen de réponse au stress
phasic_std	phasic	statistique	écart type	variabilité des réponses phasiques
phasic_min	phasic	statistique	valeur minimale	niveau le plus bas observé
phasic_max	phasic	statistique	valeur maximale	intensité maximale d'une réponse SCR
phasic_skew	phasic	distribution	asymétrie du signal	détecte la présence de pics
phasic_kurt	phasic	distribution	kurtosis (pics extrêmes)	mesure les réponses fortes
phasic_q25	phasic	quantile	25e percentile	distribution robuste
phasic_q50	phasic	quantile	médiane	valeur centrale robuste
phasic_q75	phasic	quantile	75e percentile	dispersion du signal

tonic_mean	tonic	statistique	moyenne tonic	niveau physiologique de base
tonic_std	tonic	statistique	écart type tonic	variation lente du niveau EDA
tonic_min	tonic	statistique	minimum tonic	niveau basal minimal
tonic_max	tonic	statistique	maximum tonic	niveau basal maximal
tonic_skew	tonic	distribution	asymétrie tonic	variation asymétrique du niveau basal
tonic_kurt	tonic	distribution	kurtosis tonic	détecte variations extrêmes
tonic_q25	tonic	quantile	25e percentile	distribution tonic
tonic_q50	tonic	quantile	médiane tonic	valeur centrale
tonic_q75	tonic	quantile	75e percentile	dispersion tonic
eda_mean	EDA	statistique	moyenne signal brut	niveau global de conductance
eda_std	EDA	statistique	écart type	variabilité globale
eda_min	EDA	statistique	minimum	valeur la plus basse
eda_max	EDA	statistique	maximum	valeur la plus haute
eda_skew	EDA	distribution	asymétrie	distribution du signal
eda_kurt	EDA	distribution	kurtosis	présence de pics
eda_q25	EDA	quantile	25e percentile	distribution robuste
eda_q50	EDA	quantile	médiane	valeur centrale
eda_q75	EDA	quantile	75e percentile	dispersion
phasic_auc	phasic	dynamique	aire sous la courbe	activité totale du système sympathique
phasic_rms	phasic	dynamique	root mean square	énergie du signal
tonic_slope	tonic	dynamique	pente du signal tonic	évolution du niveau physiologique
spectral_centroid	phasic	fréquentiel	centre spectral	distribution des fréquences du signal
phasic_iqr	phasic	dispersion	interquartile range	variabilité robuste
tonic_iqr	tonic	dispersion	interquartile range	dispersion tonic

eda_iqr	EDA	dispersion	interquartile range	dispersion globale
phasic_entropy	phasic	complexité	entropie	complexité des réponses phasiques
tonic_entropy	tonic	complexité	entropie	complexité tonic
eda_entropy	EDA	complexité	entropie	complexité du signal global

TABLEAU 2 : STATISTIQUES

II.2.2. Caractéristiques supplémentaires du dataset (non physiologiques)

Variable	Description
Participant	identifiant du sujet
Time_Min	temps de début de la fenêtre
Prediction	prédiction du modèle XGBoost (0 = relaxation, 1 = stress)

TABLEAU 3 : SUPPLEMENTAIRES

II.2.3. Catégories physiologiques des caractéristiques (Features)

❖ **Statistiques**

27 caractéristiques, elles décrivent :

- La forme du signal
- La variabilité physiologique

❖ **Dynamiques**

4 caractéristiques

- phasic_auc
- phasic_rms
- tonic_slope
- spectral_centroid

Elles décrivent **l'évolution temporelle du signal**.

❖ **Dispersion robuste**

3 caractéristiques

- phasic_iqr
- tonic_iqr
- eda_iqr

Moins sensibles aux **artefacts et outliers**.

❖ **Complexité du signal**

3 Caractéristiques

- phasic_entropy
- tonic_entropy
- eda_entropy

Elles mesurent **l'irrégularité physiologique**, souvent liée au stress.

II.3. Pourquoi utiliser 37 caractéristiques ?

Le modèle essaie de capturer **4 dimensions du signal EDA** :

- **niveau physiologique**
  (moyennes, min, max)
- **variabilité**
  (std, IQR)
- **dynamique temporelle**
  (AUC, RMS, slope)
- **complexité du signal**
  (entropy, spectral)

Cette combinaison améliore la **détection du stress par machine learning**.

A la fin de l'algorithme feature engineering La base de données est devenue sous cette forme :

Participant	Time_Min	Prediction	phasic_mean	phasic_std	eda_entropy	...
ID01	0	1	...	...	...	...
ID01	1	0	...	...	...	...
ID02	0	1	...	...	...	...

TABLEAU 4 : BASE DE DONNEES APRES LE TRAITEMENT DU SIGNAL EDA

Chaque ligne correspond à une fenêtre temporelle du signal d'un participant.

- **Participant :** identifie chaque sujet, utilisé pour la validation croisée par groupe (GroupKFold) afin d'éviter d'entraîner et tester sur le même participant
- **Time_min :** Temps de début de la fenêtre, permet de situer la prédiction dans le temps.
- **Prediction :** Sortie du modèle XGBoost, 0 pour relaxation et 1 pour Stress, utilisé comme label pour classification.

III Paramètres du modèle Random Forest (Classification et validation)

```
pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("rf", RandomForestClassifier(
        n_estimators=300,
        class_weight="balanced",
        random_state=42,
        n_jobs=-1
    ))
])
```

FIGURE 5 : CODE RANDOM FOREST

III.1. Pipeline

Cette étape est le **moteur** de notre projet. Le **Pipeline** est un conteneur qui attache nos outils ensemble pour qu'ils fonctionnent comme une seule unité.

La chaîne de traitement est implentée avec :

Sklearn.pipeline.Pipeline

Pourquoi utiliser un Pipeline ?

- **Automatisation** : Au lieu de nettoyer vos données (Imputer), puis d'entraîner votre modèle (RF) séparément, le pipeline fait tout d'un coup.
- **Sécurité (Data Leakage)** : Il garantit que les statistiques utilisées pour remplir les vides (comme la médiane) sont calculées uniquement sur les données d'entraînement et appliquées ensuite aux données de test, évitant ainsi que le modèle ne "triche".

Comparaison avec et sans pipeline :

Sans Pipeline (Manuel)	Avec Pipeline (Automatisé)
1. `imputer.fit(X_train)`	`pipe.fit(X_train, y_train)`
2. `X_tr_imputed = imputer.transform(X_train)`	*(Toutes les étapes sont faites en une ligne)*
3. `rf.fit(X_tr_imputed, y_train)`	
4. `X_te_imputed = imputer.transform(X_test)`	`predictions = pipe.predict(X_test)`
5. `predictions = rf.predict(X_te_imputed)`	

TABLEAU 5 : PIPELINE COMPARAISON

Structure :

Imputation → Random Forest

Etape 1 :

SimpleImputer(strategy="median")

Objectif : Remplacer les valeurs manquantes par des valeurs des médians.

Etape 2 :

RandomForest classifier

III.2. n_estimators = 300

n_estimators = 300 c'est le nombre d'arbres.

- augmente la stabilité du modèle
- réduit le surapprentissage.

III.3 class_weight = "balanced"

Cette option donne **plus d'importance aux classes minoritaires** pendant l'apprentissage.

Elle est exprimée par la formule :

$$w_i = \frac{N}{C \times n_i}$$

Où :

- **N** = nombre total d'échantillons
- **C** = nombre de classes
- **ni** = nombre d'échantillons de la classe *i*

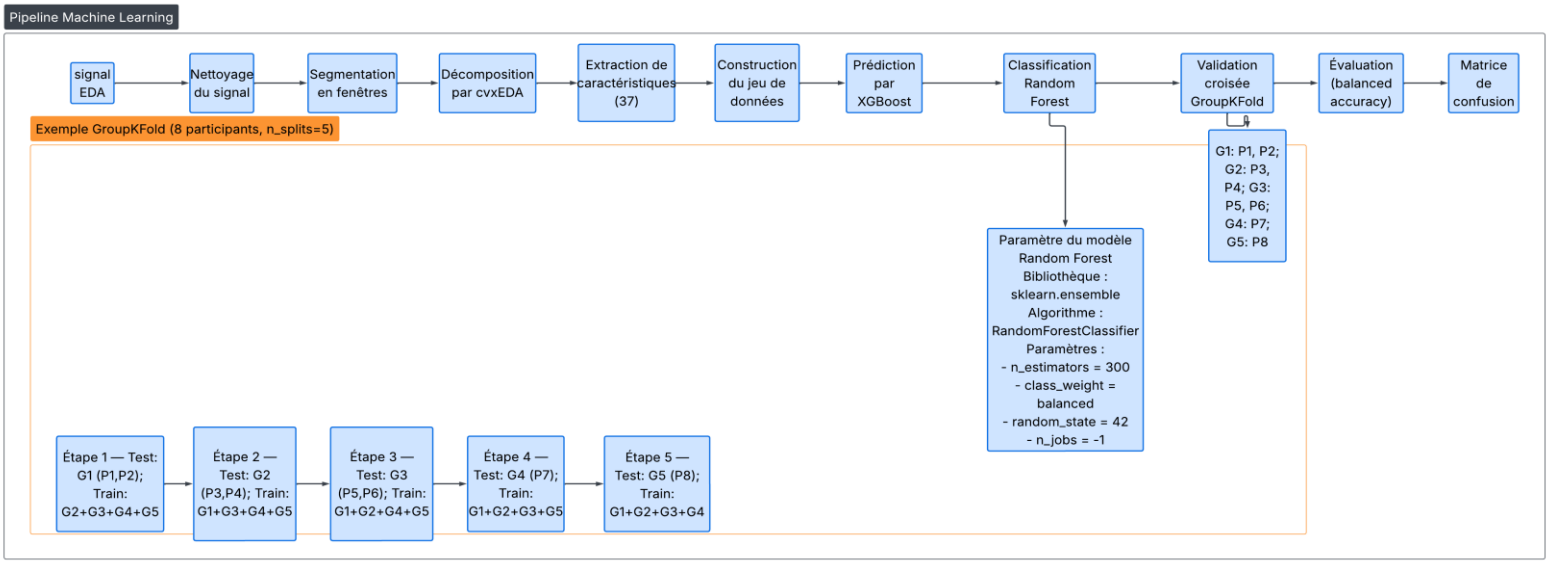


FIGURE 6 : SHEMA RANDOM FOREST

Exemple

Base de données :

Classe	Nombre
Relaxation	800
Stress	200

TABLEAU 6 BASE DE DONNEES

Total :

```
N = 1000
C = 2
```

- Relaxation :

  $w0 =$ 0.625

- Poids Stress :

$w1$=2.5

Donc :

$$\frac{w1}{w0} = 4$$

Une erreur sur stress est 4 fois plus pénalisée qu'une erreur sur relaxation ce qui compense le **déséquilibre entre les classes** (stress / relaxation).

III.4. random_state = 42

Permet de **reproduire les résultats**.

IIII.5. n_jobs = -1

Utilise **tous les cœurs CPU** pour accélérer l'entraînement.

Structure :

Imputation → Random Forest

Etape 1 :

SimpleImputer(strategy="median")

Objectif : Remplacer les valeurs manquantes par des valeurs des médians.

Etape 2 :

RandomForest classifier

Objectif et la raison sont expliqué dans la **section 2.1** paramètre du Random Forest

IV. Validation

```
# 2. Configuration de la validation croisée
# (On s'assure que 'groups' est bien défini, sinon on le crée depuis df_clean)
if 'groups' not in locals():
    groups = df_clean[col_participant].astype(str).values

n_participants = len(np.unique(groups))
gkf = GroupKFold(n_splits=min(5, n_participants))
```

C'est quoi ? On s'assure que la variable groups contient bien l'identifiant de chaque participant (ex: P1, P2, P3...).

Pourquoi ? Pour l'étape suivante, l'algorithme doit savoir quelle ligne de données appartient à quel être humain.

Les données sont séparées par participant

évite que les données d'un même participant soient dans train et test.

Code :

```
gkf = GroupKFold(n_splits=min(5, n_participants))
```

Méthode :

`GroupKFold`

Raison :

Dans la plupart des projets de Machine Learning, on utilise un `KFold` classique qui mélange les données aléatoirement. **Mais pour des données physiologiques (EDA), c'est une erreur grave.**

- **Le problème (Le mélange) :** Si j'ai 100 mesures du Participant A, et que j'en mettre 80 en entraînement et 20 en test, le modèle va "reconnaître" la signature physiologique spécifique du Participant A. Il aura d'excellents scores, mais il échouera totalement face à un nouveau Participant B.
- **La solution (`GroupKFold`) :** Cet algorithme garantit qu'un participant entier est soit dans le groupe d'entraînement, soit dans le groupe de test, mais **jamais dans les deux en même temps**.

Code :

n_splits=min (5, n_participants)

- **C'est quoi ?**

`n_splits` correspond au **nombre de divisions du dataset** pour la validation croisée.

Si :

```
n_splits = 5
```

le dataset est séparé en **5 groupes (folds)**.

> On divise les participants en 5 groupes (ou moins si vous avez moins de 5 participants).

- **Pourquoi ?**

La fonction :

```
min(a, b)
```

retourne **la plus petite valeur entre a et b**.

Donc :

```
n_splits = min(5, n_participants)
```

signifie :

- maximum **5 folds**
- mais **jamais plus que le nombre de participants**.

> -> Le modèle va s'entraîner sur 4 groupes et se tester sur le 5ème.
>
> - Il recommence l'opération 5 fois pour que chaque groupe soit testé une fois.
> - Cela donne une **moyenne de performance** beaucoup plus fiable qu'un simple test unique. Le `min()` est une sécurité technique : on ne peut pas diviser en 5 groupes si on n'a que 3 participants !

Prenons **8 participants** :

P1, P2, P3, P4, P5, P6, P7, P8

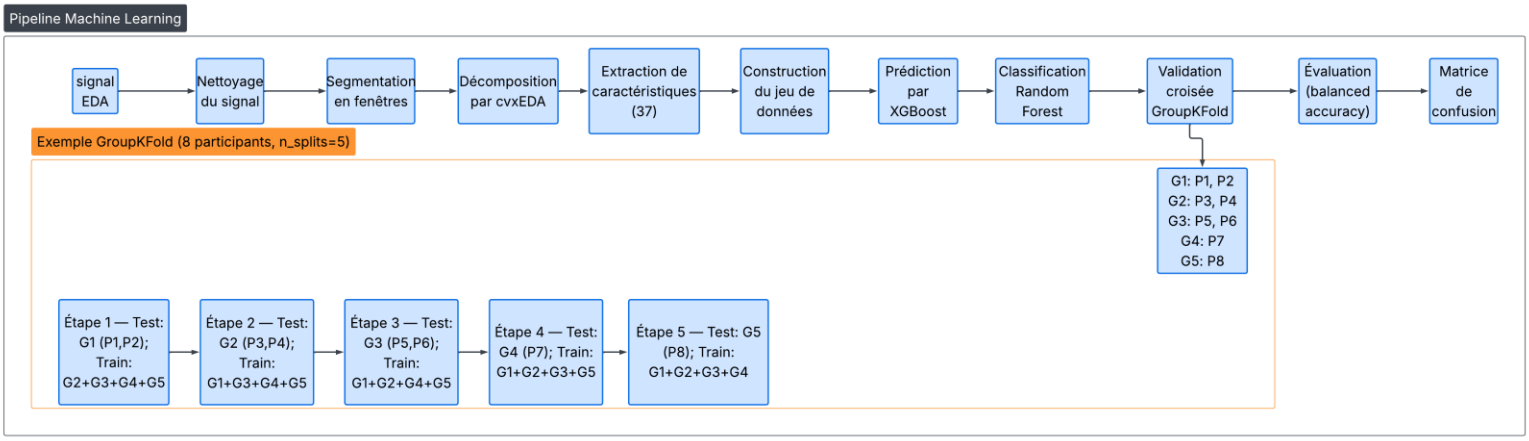
Avec `n_splits = min(5, n_participants)`
→ `min(5, 8) = 5`

Donc on fait **5 groupes (folds)**.

Comme on a 8 participants pour 5 groupes, certains groupes auront **2 participants** et d'autres **1 participant**.

Exemple de division

Groupe (Fold)	Participants
G1	P1, P2
G2	P3, P4
G3	P5, P6
G4	P7
G5	P8



Résumé

Les variables ont été choisies pour capturer 4 aspects du signal EDA :

- **niveau physiologique** → mean, min, max
- **variabilité** → std, IQR
- **forme statistique** → skew, kurtosis
- **dynamique du signal** → AUC, RMS, spectral, slope
- **Complexité** → entropy

Ce mélange de **statistiques, dynamique et fréquence** améliore la capacité du modèle à détecter le **stress à partir de l'EDA**.

V. Bilan

Le bilan s’est calculé comme ce qui est marqué sur la figure

```python
# ========================
# 5) BILAN & GRAPHIQUE
# ========================
print("\n" + "=" * 30)
print(f"SCORE FINAL MOYEN : {np.mean(results):.3f} ± {np.std(results):.3f}")
print("=" * 30)
```

C'est quoi ? La moyenne des scores obtenus sur tous les plis (folds), accompagnée de l'écart-type (la variation).

Pourquoi ? Un score moyen de 0.85 avec un écart-type de 0.02 signifie que le modèle est stable. Si l'écart-type est de 0.20, cela signifie que le modèle fonctionne très bien sur certains participants mais échoue totalement sur d'autres.

Importance des features est classifié selon cette logique :

```python
# --- A. TABLEAU D'IMPORTANCE DES VARIABLES ---
if hasattr(pipe.named_steps["rf"], "feature_importances_"):
    importance = pipe.named_steps["rf"].feature_importances_
    feat_imp = pd.Series(importance, index=X_cols).sort_values(ascending=False)
    print("\nTop 5 des variables explicatives :")
    print(feat_imp.head(5))
```

C'est quoi ? Le Random Forest regarde quelles colonnes (ex: tonic_mean ou phasic_peaks) l'ont le plus aidé à prendre les bonnes décisions.

Pourquoi ? C'est la partie "explicable" de l'IA. Cela permet de dire : "Le stress est principalement détecté par l'augmentation de l'amplitude phasique". C'est ce qui donne une valeur scientifique à votre travail, au-delà de la simple prédiction.

Bibliographie

[1] Gashi et al. detection of artifacts in Ambulotary electrodermal activity data

[2] Greco et al. 2015

[3] Hasan Ahmed Salman et al 2024 Random Forest Algorithm Overview